

Automating Workloads

10

Topics

- | | |
|----|------------------------------------|
| 01 | Scheduling Queries |
| 02 | Scripting in BigQuery |
| 03 | Stored Procedures |
| 04 | Integration with Big Data Products |



A blue square containing the white text '01'. To the right of the square is a yellow downward-pointing triangle.

01

Scheduling Queries

Common pitfall: “My dashboard takes too long to load!”

Where is your dashboard pulling data from?

Dashboard powered by
Tables (stored data)

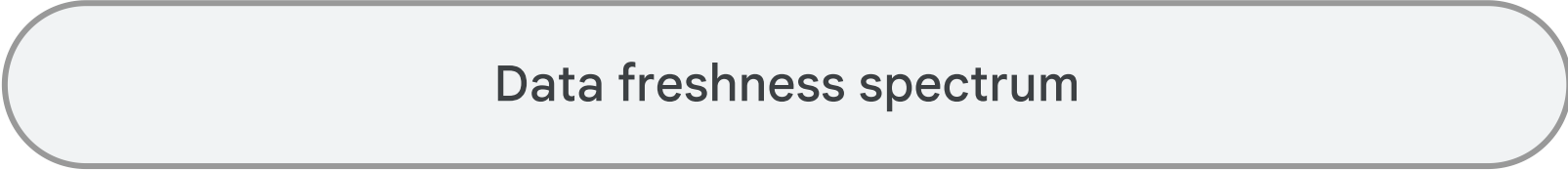
versus

Dashboard powered by
Views (stored query)

Common pitfall: “My dashboard takes too long to load!”

Create permanent reporting **tables** that pre-calculate fields for you to visualize.

Power your dashboard entirely off SQL **views** that query the raw data each time.



Data freshness spectrum

Can be **stale** but really **fast to visualize**.

Fresh but can be performance **intensive**.

Common solution: Create reporting tables that refresh on a regular cadence

Create permanent reporting **tables** that pre-calculate fields for you to visualize.

Create reporting **tables that automatically refresh** at regular intervals.

Power your dashboard entirely off SQL **views** that query the raw data each time.

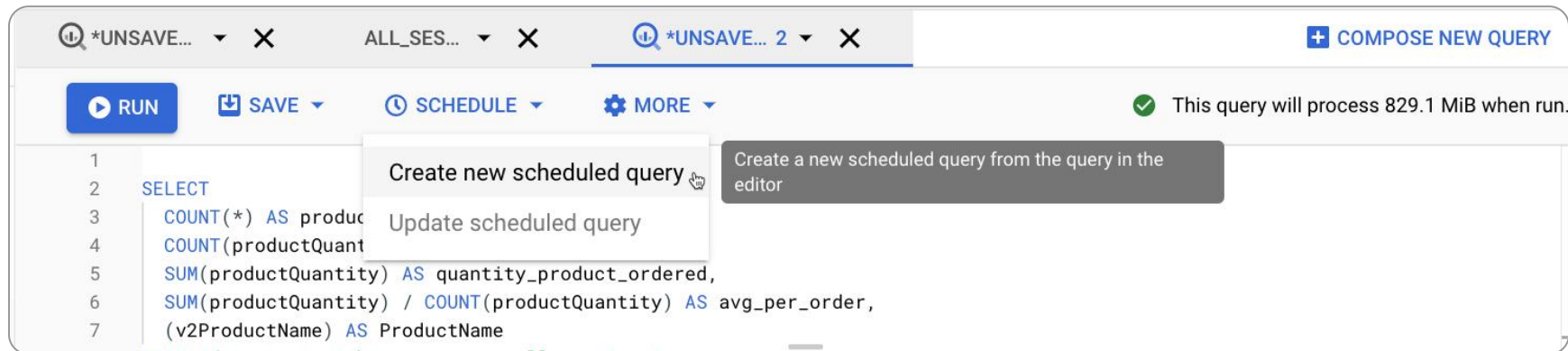
Data freshness spectrum

Can be **stale** but really **fast to visualize**.

Create **automatically updated** reporting tables.

Fresh but can be performance **intensive**.

In BigQuery, you can create scheduled queries that will save your results periodically into a table



Automate the execution of queries based on a schedule

New scheduled query

Details and schedule

Name for scheduled query

Schedule options

Repeats

Daily

☒ Start now ☐ Schedule start time

☒ End never ☐ Schedule end time

 This schedule will run Every day at 17:46 Europe/London

Destination for query results

 A destination table is required to save scheduled query options.

Project name Dataset name

qwiklabs-gcp-c710de4945fa7b9c champions_workshop_models

Table name

scheduled_query_output

Destination table write preference

☒ Append to table
☐ Overwrite table

Notification options

☐ Send email notifications ?

Schedule Cancel

You can have multiple saved scheduled queries

Reporting daily query:
Runs **daily** at 4 AM

Reporting weekly query:
Runs **Friday** at 4 AM

Reporting archival query:
Runs **monthly** at 4 AM

New scheduled query

Details and schedule

Name for scheduled query

Schedule options

☒ Start now ☐ Schedule start time

Repeats Start date and run time

Daily 4/22/19, 2:53 PM PDT

 This schedule will run Every day at 14:53 America/Los Angeles

Destination for query results

 A destination table is required to save scheduled query options.

Project name Dataset name

data-to-insights ecommerce

Table name

Letters, numbers, and underscores allowed

Destination table write preference

☒ Append to table ☐ Overwrite table

Notification options

☐ Send email notifications 

Scheduled query: UI options

Google Cloud Platform tw-project-D Search products and resources

BigQuery Scheduled qu... **RUN HISTORY** **CONFIGURATION** EDIT DELETE DISABLE **SCHEDULE BACKFILL** MORE

cross_project_scheduled_query

Schedule (UTC) every 24 hours Target date for next run April 23, 2021 at 8:54:00 PM UTC

Filter Filter transfer runs

	Run date	Schedule time	Summary
○	April 22, 2021	April 22, 2021 at 1:54:00 PM UTC-7	The transfer run has completed successfully.
●	April 21, 2021	April 21, 2021 at 1:54:00 PM UTC-7	The transfer run has completed successfully.
○	April 20, 2021	April 20, 2021 at 1:54:00 PM UTC-7	The transfer run has completed successfully.
○	April 19, 2021	April 19, 2021 at 1:54:00 PM UTC-7	The transfer run has completed

Run details

Resource name projects/909176438758/locations/us/transferConfigs/60bd5797-0000-2cb6-90e6-94eb2c07a8f2/runs/61164756-0000-274f-a681-f403043ae20

Started April 21, 2021 at 8:54:02 PM UTC

Duration 1 min 13 sec

Logs

Filter Filter logs

Log time	Message
1:55:15 PM	Summary: succeeded 1 jobs, failed 0 jobs.
1:55:04 PM	Job scheduled_query_61164756-0000-274f-a681-f403043ae200 (table cross_project_scheduled_query) completed successfully.
1:54:03 PM	Job scheduled_query_61164756-0000-274f-a681-f403043ae200 (table cross_project_scheduled_query) started.
1:54:02 PM	Starting to process the query job with no parameters.
1:54:00 PM	Dispatched run to data source with id 2301993672050926

Scheduled query

```
bq query \
--use_legacy_sql=false \
--project_id=<compute_project> \
--destination_table=mydataset.mytable \
--display_name='Test Scheduled Query' \
--schedule='every 24 hours' \
--replace=true \
'SELECT COUNT(*) FROM `bigquery-public-data.new_york_citibike.citibike_trips`
WHERE start_station_name LIKE "%Broadway%"'
```

```
bq mk \
--transfer_config \
--project_id=<compute_project> \
--location=US \
--target_dataset=mydataset \
--display_name='Test Scheduled Query' \
--params='{ "query": "SELECT COUNT(*) FROM `bigquery-public-
data.new_york_citibike.citibike_trips`
WHERE start_station_name LIKE `%Broadway%`"
, "destination_table_name_template": "mytable"
, "write_disposition": "WRITE_TRUNCATE"}' \
--data_source=scheduled_query
```

Details and schedule

Name for scheduled query

Test Scheduled Query

Schedule options

Choose frequency, time and time zone (local time zone is selected by default) and BigQuery will convert and schedule the query in UTC time.

Repeats

Daily

☒ Start now
 ☐ Schedule start time

☐ End never
 ☒ Schedule end time

End date

4/23/21, 8:21 AM PDT

 This schedule will run Every day at 15:21 UTC, starting now and ending Fri Apr 23 2021

Destination for query results

 A destination table is required to save scheduled query options.

Project name

tw-project-0

Dataset name

dts_test

Table name

stackoverflow

Destination table partitioning field

Destination table write preference

☐ Append to table
 ☒ Overwrite table

Advanced options

Notification options

☒ Send email notifications

Schedule

Cancel

Scheduled query: Parameterization

run_time (UTC)	Templated parameter	Output destination table name
2018-02-15 00:00:00	mytable	mytable
2018-02-15 00:00:00	mytable_{run_time "%"Y%m%d"}	mytable_20180215
2018-02-15 00:00:00	mytable_{run_time+25h "%"Y%m%d"}	mytable_20180216
2018-02-15 00:00:00	mytable_{run_time-1h "%"Y%m%d"}	mytable_20180214
2018-02-15 00:00:00	mytable_{run_time+1.5h "%"Y%m%d%H"} or mytable_{run_time+90m "%"Y%m%d%H"}	mytable_2018021501
2018-02-15 00:00:00	{run_time+97s "%"Y%m%d"}_mytable_{run_time+97s "%"H%M%S"}	20180215_mytable_000137

Scheduled query: Benefit BI performance

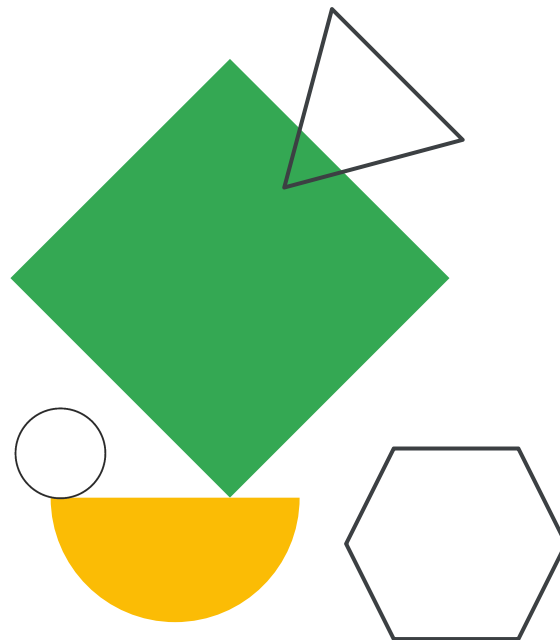


Data Analysis and Visualization Tools

Demo

Scheduling queries with
parameters

Using the @run_time parameter





Scripting in BigQuery

Need custom transformations? BigQuery supports user-defined functions in SQL, JavaScript, and scripting

Query editor

```
1 CREATE TEMP FUNCTION multiplyInputs(x FLOAT64, y FLOAT64)
2 RETURNS FLOAT64
3 LANGUAGE js AS """
4   return x*y;
5   """;
6 WITH numbers AS
7   (SELECT 1 AS x, 5 as y
8    UNION ALL
9    SELECT 2 AS x, 10 as y
10   UNION ALL
11   SELECT 3 as x, 15 as y)
12 SELECT x, y, multiplyInputs(x, y) as product
13 FROM numbers;
```

Run

Save query

Save view

Schedule query

More

Query complete (2.5 sec elapsed, 0 B processed)

Job informationResultsJSONExecution details

Row	x	y	product
1	1	5	5.0
2	2	10	20.0
3	3	15	45.0

What are user-defined functions?

A **user-defined function** (UDF) lets you create a function by using a SQL expression or JavaScript code. A UDF accepts columns of input, performs actions on the input, and returns the result of those actions as a value.

Temporary

To create: [CREATE FUNCTION](#)

To delete: expire as soon as the query finishes.

(The [DROP FUNCTION](#) statement is only supported for temporary UDFs in [multi-statement queries](#) and [procedures](#).)

Persistent

To create: [CREATE FUNCTION](#)

To delete: [DROP FUNCTION](#)

Temporary SQL UDF example

```
CREATE TEMP FUNCTION
```

```
AddFourAndDivide(x INT64, y INT64)
```

```
RETURNS FLOAT64
```

```
AS ((x + 4) / y);
```

```
SELECT val, AddFourAndDivide(val, 2)
```

```
FROM UNNEST([2,3,5,8,12]) AS val;
```

Row	val	f0_
1	2	3.0
2	3	3.5
3	5	4.5
4	8	6.0
5	12	8.0

Persistent SQL UDF example

```
CREATE FUNCTION
```

```
demo.AddFourAndDivide(x INT64, y INT64)
```

```
RETURNS FLOAT64
```

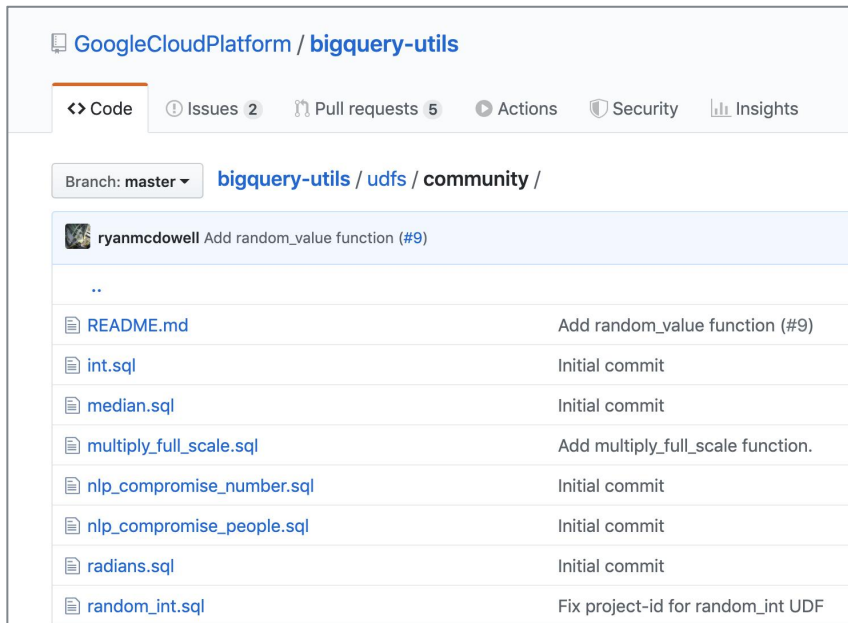
```
AS ((x + 4) / y);
```

```
SELECT val, demo.AddFourAndDivide(val,  
2)
```

```
FROM UNNEST([2,3,5,8,12]) AS val;
```

Row	val	f0_
1	2	3.0
2	3	3.5
3	5	4.5
4	8	6.0
5	12	8.0

You can persist and share your UDF objects with other team members or publically



The BigQuery team has a public GitHub repo for common user-defined functions:

<https://github.com/GoogleCloudPlatform/bigquery-utils/tree/master/udfs/community>

Temporary JavaScript UDF example

```
CREATE TEMP FUNCTION multiplyInputs(x FLOAT64, y  
FLOAT64)
```

```
RETURNS FLOAT64
```

```
LANGUAGE js AS r"""  
  return x*y;  
""";
```

```
WITH numbers AS
```

```
(SELECT 1 AS x, 5 as y
```

```
UNION ALL
```

```
SELECT 2 AS x, 10 as y
```

```
UNION ALL
```

```
SELECT 3 as x, 15 as y)
```

```
SELECT x, y, multiplyInputs(x, y) as product
```

```
FROM numbers;
```

Row	x	y	product
1	1	5	5.0
2	2	10	20.0
3	3	15	45.0

BigQuery data type	JavaScript data type
ARRAY	ARRAY
BOOL	BOOLEAN
BYTES	base64-encoded STRING
FLOAT64	NUMBER
NUMERIC, BIGNUMERIC	If a NUMERIC or BIGNUMERIC value can be represented exactly as an IEEE 754 floating-point  value and has no fractional part, it is encoded as a Number. These values are in the range $[-2^{53}, 2^{53}]$. Otherwise, it is encoded as a String.
STRING	STRING
STRUCT	OBJECT where each STRUCT field is a named field
TIMESTAMP	DATE with a microsecond field containing the microsecond fraction of the timestamp
DATE	DATE



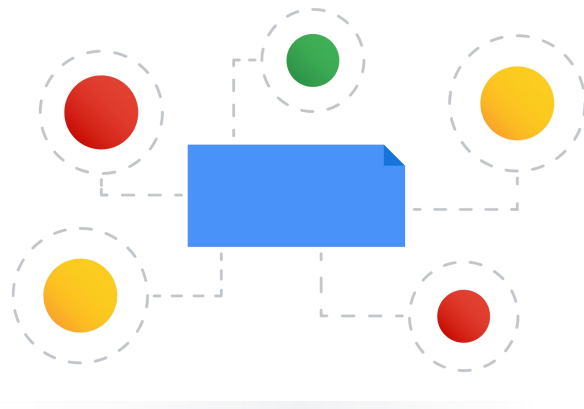
03

Stored Procedures

What are stored procedures ?

A [stored procedure](#) is a collection of statements that can be called from other queries or other stored procedures. A procedure can take input arguments and return values as output.

Stored procedures support [procedural language statements](#), which let you do things like define variables and implement control flow.



How to create stored procedures

```
CREATE PROCEDURE
dataset_name.procedure_name
BEGIN
-- statements here
END
```

```
CALL procedure_name
(procedure_argument[, ..])
```

- Use CREATE PROCEDURE statement.
- Define the procedure_name.
- Add the body of the procedure between BEGIN and END statement.
- Use CALL statement to run the procedure.

Example 1

```
CREATE OR REPLACE PROCEDURE
Scheduling_demo.test()
BEGIN
SELECT "Hello World!" AS response;
END;
CALL Scheduling_demo.test()
```

Row	response
1	Hello World!

Status	End time	SQL	Stages completed	Bytes processed	Action
✓	4:09 PM [1:1]	CREATE OR REPLACE PROCEDURE Scheduling_demo.test() BEGIN SELECT "Hello World!" AS response; END	0	0 B	VIEW RESULTS
✓	4:09 PM Procedure	SELECT "Hello World!" AS response	1	0 B	VIEW RESULTS

Example 2

```
CREATE OR REPLACE PROCEDURE
Scheduling_demo.test2(name STRING)
BEGIN
SELECT FORMAT("Hello %s!", name) AS
response;
END;
CALL Scheduling_demo.test2("Google")
```

Row	response
1	Hello Google!

Status	End time	SQL	Stages completed	Bytes processed	Action
✓	4:12 PM [1:1]	CREATE OR REPLACE PROCEDURE Scheduling_demo.test2(name STRING) BEGIN SELECT FORMAT("Hello %s!", ...	0	0 B	VIEW RESULTS
✓	4:12 PM Procedure	SELECT FORMAT("Hello %s!", name) AS response	1	0 B	VIEW RESULTS

Example 3

```
CREATE OR REPLACE PROCEDURE
`retail.add_new_name`(target_first_name STRING, target_last_name STRING)
BEGIN
IF EXISTS(SELECT 1 FROM retail.unique_customer_names_emails
WHERE firstname = target_first_name and lastname = target_last_name) THEN
SELECT CONCAT('Name ', target_first_name, " ", target_last_name , " is already in database.");
ELSE
SELECT CONCAT('Did not find name ', target_first_name, " ", target_last_name , '. Adding new name.');
```

```
INSERT retail.unique_customer_names_emails(firstname, lastname, email)
VALUES (target_first_name, target_last_name, "To Be Added");
END IF;
END;
CALL retail.add_new_name("Emma", "Le");
```

Row	f0_
1	Name Emma Le is already in database.

System procedures reference

● **BQ.JOBS.CANCEL**

- Cancels a running job.
- Specify the job:
`'[project_id.]job_id'`
- Must run the procedure in the same region as the job.

```
CALL BQ.JOBS.CANCEL(job);
```

● **BQ.REFRESH_MATERIALIZED_VIEW**

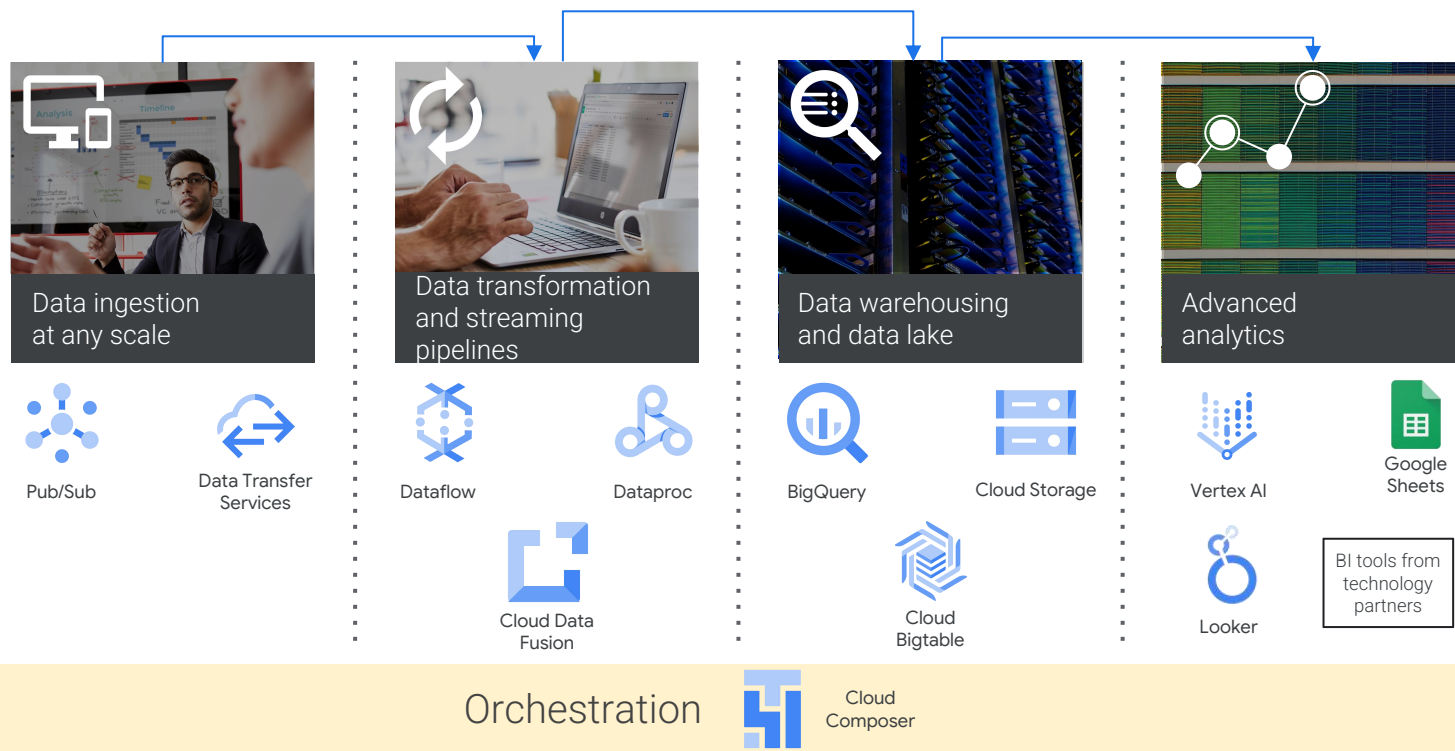
- Refreshes a materialized view.
- Specify the materialized view:
`'[project_id.]dataset.table'`

```
CALL BQ.REFRESH_MATERIALIZED_VIEW(view_name);
```



Integration with Big Data Products

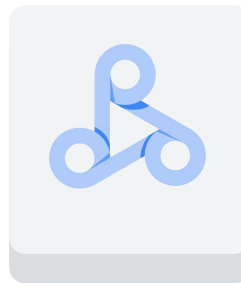
Google Cloud offers a range of ETL tools



What if your data is not usable in its original form?



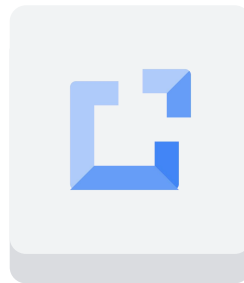
Data processing



Dataproc



Dataflow

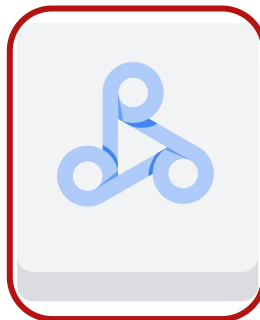


Cloud Data Fusion

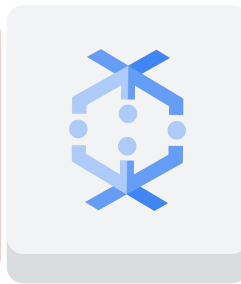
What if your data is not usable in its original form?



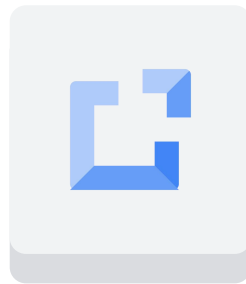
Data processing



Dataproc

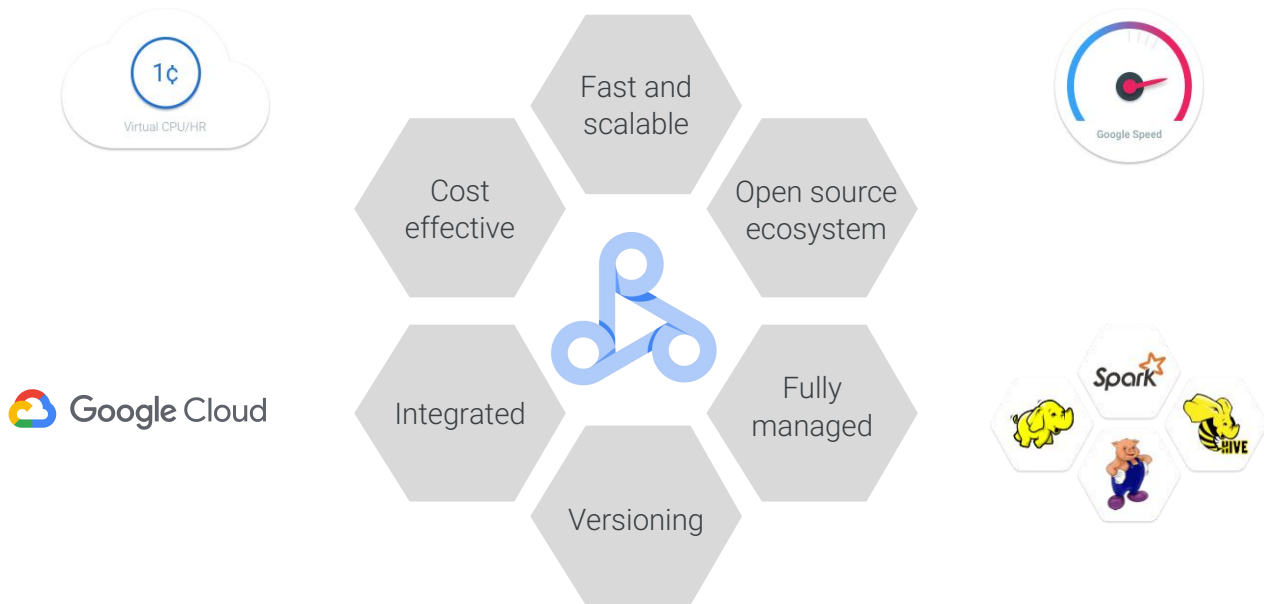


Dataflow



Cloud Data
Fusion

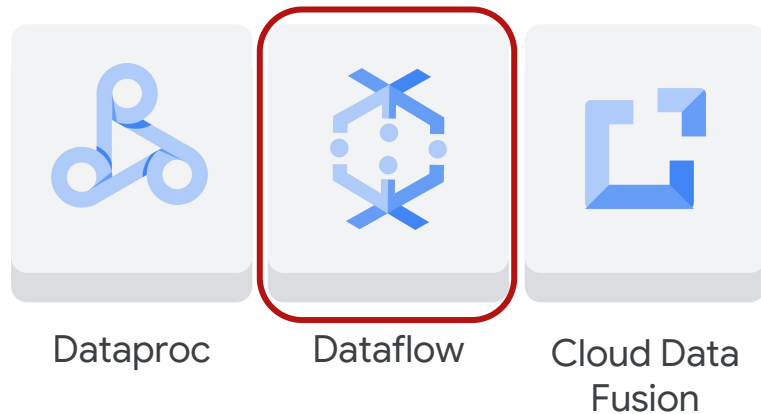
Dataproc is a managed service for batch processing, querying, streaming, and ML



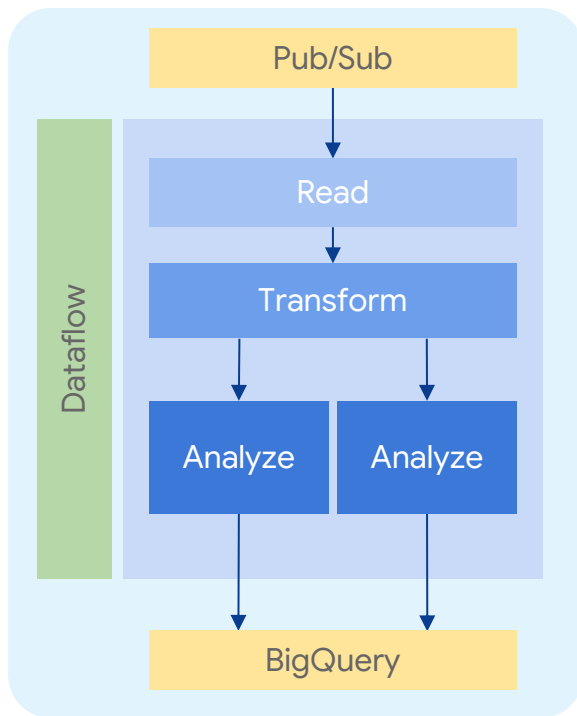
What if your data is not usable in its original form?



Data processing



Prep and filter your data before it is written to BigQuery



Dataflow runners: Two implementations with shared code

Batch

Supports only bounded sources/sinks

Finishes when pipeline is done

High throughput (low pipeline latency)

Thousands of work items



Streaming

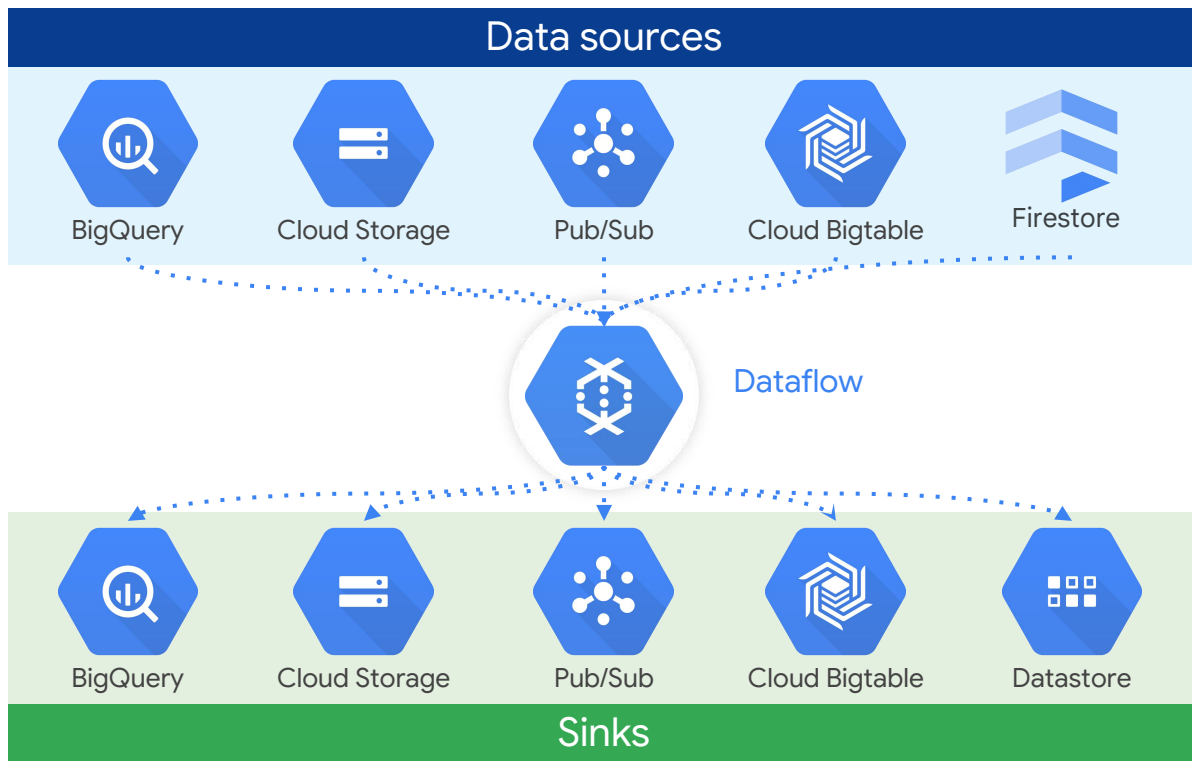
Bounded and unbounded sources/sinks

Runs indefinitely (needs cancel)

Low record latency

Billions of work items (less batching)

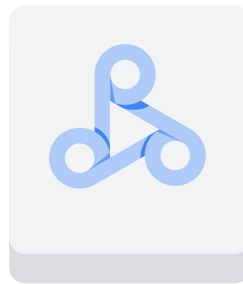
Data sources and sinks for Dataflow



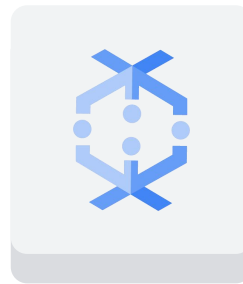
What if your data is not usable in its original form?



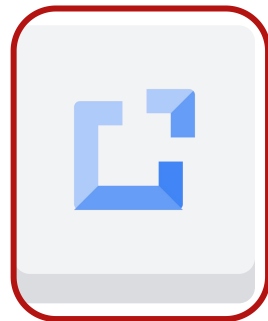
Data processing



Dataproc

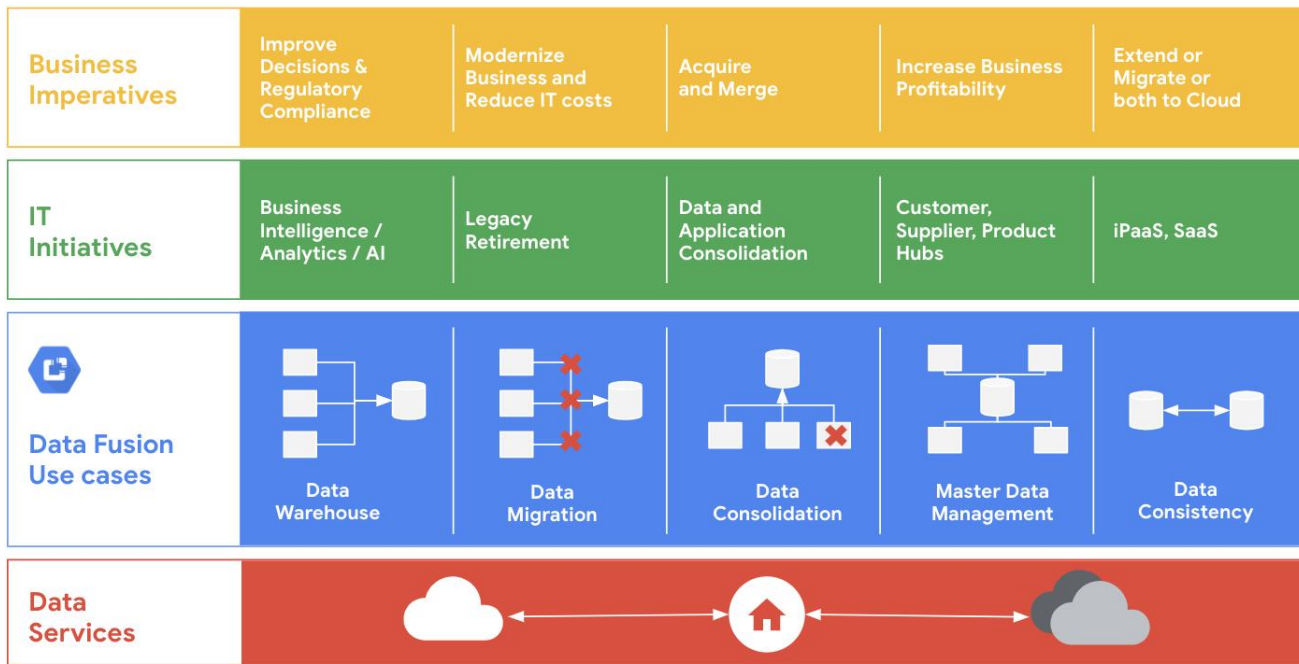


Dataflow

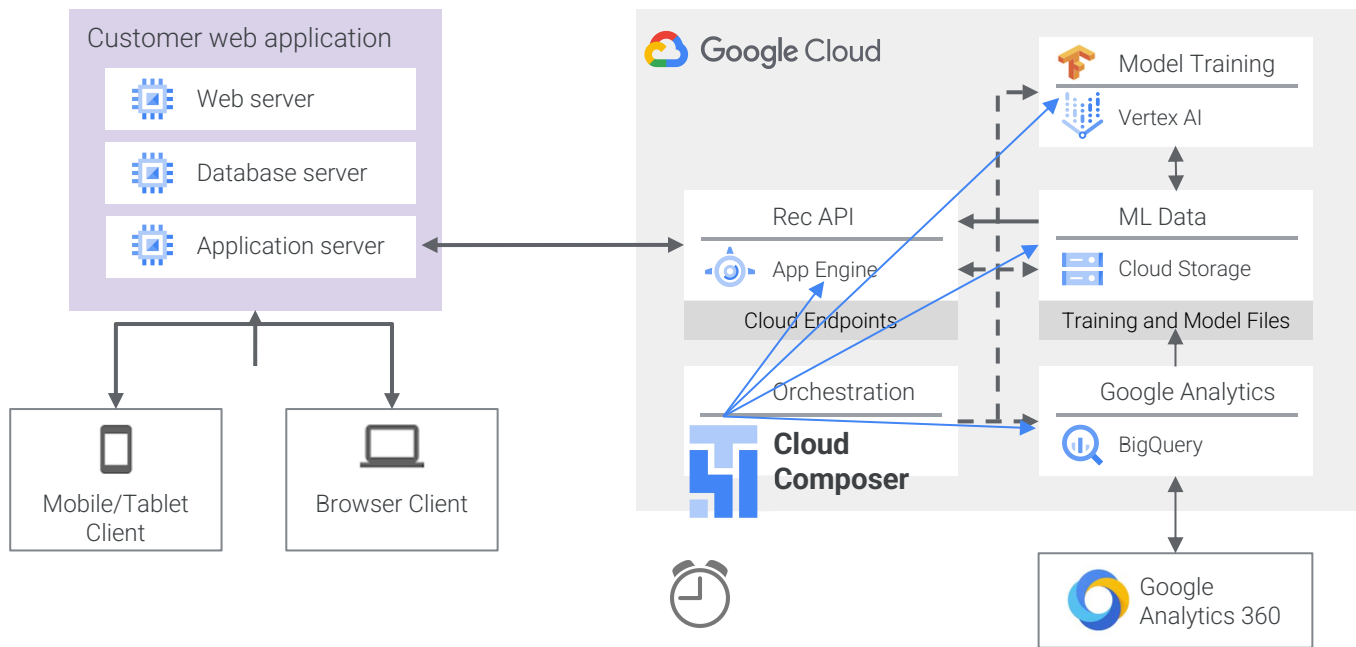


Cloud Data Fusion

Build and manage data pipelines quickly with Cloud Data Fusion

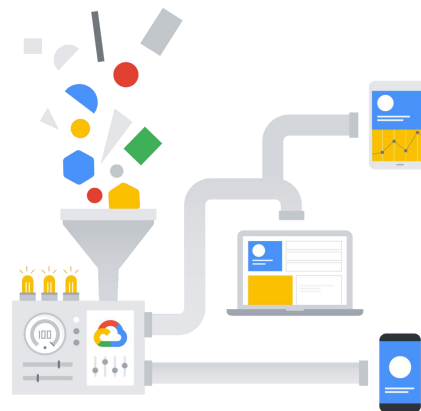


Cloud Composer (managed Apache Airflow) is used to orchestrate production workflows



Cloud Composer key features

- Hybrid and multi-cloud
- Open source
- Easy orchestration

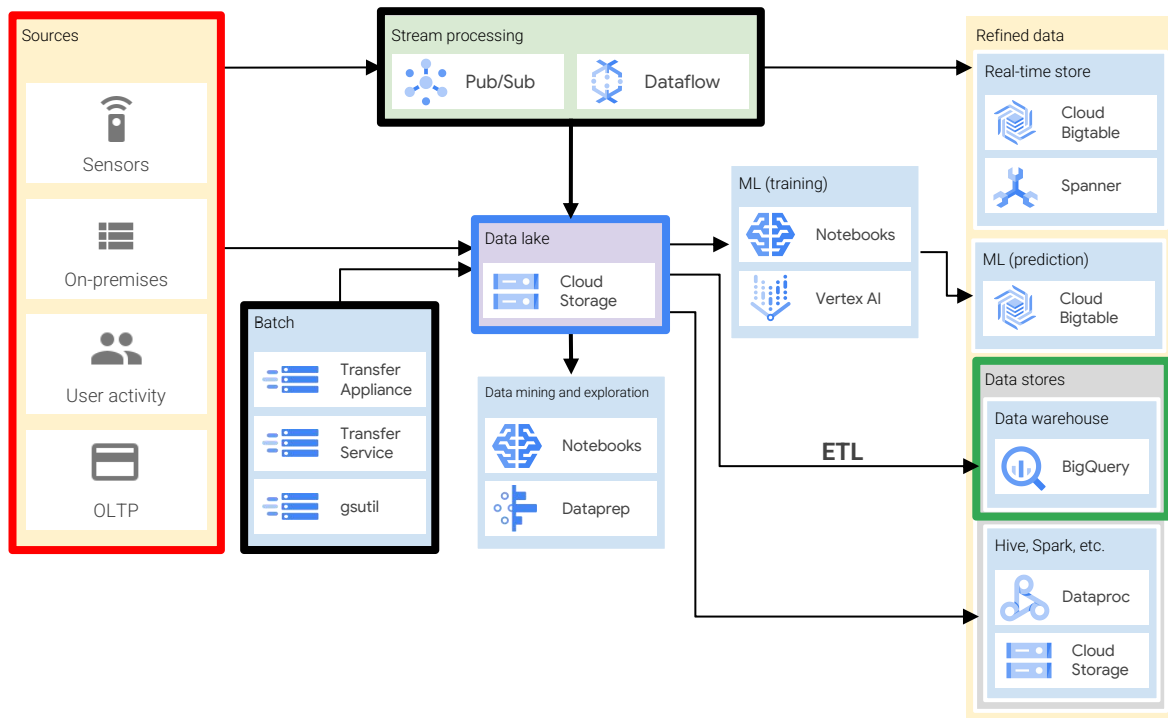


[All features](#)

Concept review

1. **Data sources**
2. **Data lake**
3. **Data pipelines**
4. **Data warehouse**

5. Used for ML and analytics workloads



Google Cloud processing options (1/2)



Dataflow



Dataproc

Recommended for:	New data processing pipelines, unified batch and streaming	Existing Hadoop/Spark applications, machine learning/data science ecosystem, large-batch jobs, preemptible VMs
Use cases:	Advertisers/contents, streaming video providers, real-time inventory	Data analysis, do development on open-source stack, pay per usage
Fully managed:	Yes	No
Expertise:	Apache Beam	Hadoop, Hive, Pig, Apache Big Data ecosystem, Spark, Flink, Presto, Druid
Autoscaling:	Yes	Yes
Interface:	Google Cloud SDK, Google Cloud console, Dataflow API	Google Cloud SDK, Google Cloud console, SSH, Zeppelin and Jupyter notebooks, Workflow templates, Dataproc API

Google Cloud processing options (2/2)



Cloud Dataprep



Cloud Data Fusion

Recommended for:	Data wrangling and preparation	New data processing pipelines
Use cases:	Cleaning and preparing structured and unstructured data for exploration, analysis, reporting, and machine learning	Visual point-and-click interface that enables the code-free development of ETL pipelines
Fully managed:	Yes: Dataflow backend	Yes: ephemeral Dataproc backend
Expertise:	Data analytics	Data analytics
Interface:	Dedicated UI	Dedicated UI

Questions?

